

API 规范：用户登录与认证

1. 通用约定

基础路径： `/api/v1`

认证规则：需要登录的接口通过 `Authorization` 请求头传递 JWT。

认证方式：

```
Authorization: Bearer <JWT_TOKEN>
```

统一成功响应：

业务码： `SUCCESS`

```
{
  "code": "SUCCESS",
  "message": "OK",
  "data": {},
  "timestamp": "yyyy-mm-ddThh:mm:ss+08:00",
  "traceId": "req-0001"
}
```

统一失败响应：

错误码： `VALIDATION_FAILED`

```
{
  "code": "VALIDATION_FAILED",
  "message": "请求参数不合法",
  "details": "email 格式不正确",
  "timestamp": "yyyy-mm-ddThh:mm:ss+08:00",
  "traceId": "req-0002"
}
```

安全约束：

密码只允许通过 HTTPS 提交，服务端只保存 `passwordHash`，不保存明文密码。

登录成功后返回访问令牌和刷新令牌，后续业务接口使用访问令牌。

管理员权限由 `role` 字段和后端权限校验共同控制，前端展示结果不能作为权限依据。

2. 业务码/错误码

业务码	HTTP 状态码	说明
SUCCESS	200 / 201 / 204	请求成功

错误码	HTTP 状态码	说明
VALIDATION_FAILED	400	参数缺失或格式错误
EMAIL_ALREADY_REGISTERED	409	邮箱已被注册
USERNAME_ALREADY_TAKEN	409	用户名已被占用
INVALID_CREDENTIALS	401	邮箱或密码错误
UNAUTHORIZED	401	未登录或访问令牌无效
TOKEN_EXPIRED	401	访问令牌已过期
ACCOUNT_DISABLED	403	用户账号被禁用
FORBIDDEN	403	当前用户无权限
USER_NOT_FOUND	404	用户不存在
PASSWORD_POLICY_VIOLATION	400	密码不符合安全策略
INTERNAL_ERROR	500	服务器内部错误

3. 用户注册

接口： `POST /api/v1/auth/register`

功能：创建平台自有账号。注册不依赖 GitHub OAuth。

权限：公开访问。

请求参数

请求头：

参数名	类型	必填	说明
Content-Type	String	是	application/json

请求体：

参数名	类型	必填	说明
username	String	是	用户名，长度 3-32，全平台唯一
email	String	是	登录邮箱，全平台唯一
password	String	是	登录密码，长度 8-64，需包含字母和数字
role	String	否	初始角色，默认 BEGINNER

请求示例：

```
POST /api/v1/auth/register
Content-Type: application/json
```

```
{
  "username": "alice",
  "email": "alice@example.com",
  "password": "GitGuild2026",
  "role": "BEGINNER"
}
```

成功响应

HTTP 状态码： 201 Created

业务码： SUCCESS

```
{
  "code": "SUCCESS",
  "message": "注册成功",
  "data": {
    "userId": 10001,
    "username": "alice",
    "email": "alice@example.com",
    "role": "BEGINNER",
    "status": "ACTIVE",
    "createdAt": "2026-05-15T20:00:00+08:00"
  },
  "timestamp": "2026-05-15T20:00:00+08:00",
  "traceId": "req-auth-0001"
}
```

失败响应

HTTP 状态码: 409 Conflict

错误码: EMAIL_ALREADY_REGISTERED

```
{
  "code": "EMAIL_ALREADY_REGISTERED",
  "message": "邮箱已被注册",
  "details": "alice@example.com 已存在对应账号",
  "timestamp": "2026-05-15T20:00:00+08:00",
  "traceId": "req-auth-0002"
}
```

4. 用户登录

接口: POST /api/v1/auth/login

功能: 校验邮箱和密码，签发访问令牌与刷新令牌。

权限: 公开访问。

请求参数

请求头:

参数名	类型	必填	说明
Content-Type	String	是	application/json

请求体:

参数名	类型	必填	说明
email	String	是	登录邮箱
password	String	是	登录密码
remember	Boolean	否	是否延长登录有效期

请求示例:

```
POST /api/v1/auth/login
Content-Type: application/json
```

```
{
  "email": "alice@example.com",
  "password": "GitGuild2026",
  "remember": true
}
```

成功响应

HTTP 状态码: 200 OK

业务码: SUCCESS

```
{
  "code": "SUCCESS",
  "message": "登录成功",
  "data": {
    "accessToken": "jwt-access-token",
    "refreshToken": "jwt-refresh-token",
    "tokenType": "Bearer",
    "expiresIn": 7200,
    "user": {
      "userId": 10001,
      "username": "alice",
      "email": "alice@example.com",
      "role": "BEGINNER",
      "status": "ACTIVE"
    }
  },
  "timestamp": "2026-05-15T20:05:00+08:00",
  "traceId": "req-auth-0003"
}
```

失败响应

HTTP 状态码: 401 Unauthorized

错误码: INVALID_CREDENTIALS

```
{
  "code": "INVALID_CREDENTIALS",
  "message": "邮箱或密码错误",
  "details": "登录凭据校验失败",
  "timestamp": "2026-05-15T20:05:00+08:00",
  "traceId": "req-auth-0004"
}
```

5. 刷新访问令牌

接口： `POST /api/v1/auth/refresh`

功能：使用刷新令牌换取新的访问令牌。

权限：持有有效刷新令牌的用户。

请求参数

请求头：

参数名	类型	必填	说明
<code>Content-Type</code>	<code>String</code>	是	<code>application/json</code>

请求体：

参数名	类型	必填	说明
<code>refreshToken</code>	<code>String</code>	是	刷新令牌

请求示例：

```
POST /api/v1/auth/refresh
Content-Type: application/json
```

```
{
  "refreshToken": "jwt-refresh-token"
}
```

成功响应

HTTP 状态码： `200 OK`

业务码： `SUCCESS`

```
{
  "code": "SUCCESS",
  "message": "令牌已刷新",
  "data": {
    "accessToken": "new-jwt-access-token",
    "tokenType": "Bearer",
    "expiresIn": 7200
  },
  "timestamp": "2026-05-15T20:10:00+08:00",
  "traceId": "req-auth-0005"
}
```

失败响应

HTTP 状态码: 401 Unauthorized

错误码: TOKEN_EXPIRED

```
{
  "code": "TOKEN_EXPIRED",
  "message": "刷新令牌已过期",
  "details": "请重新登录",
  "timestamp": "2026-05-15T20:10:00+08:00",
  "traceId": "req-auth-0006"
}
```

6. 获取当前登录用户

接口: GET /api/v1/users/me

功能: 获取当前登录用户的身份、角色和账号状态。

权限: 登录用户。

请求参数

请求头:

参数名	类型	必填	说明
Authorization	String	是	登录令牌, 格式为 Bearer <JWT_TOKEN>

请求示例:

```
GET /api/v1/users/me
Authorization: Bearer <JWT_TOKEN>
```

成功响应

HTTP 状态码: 200 OK

业务码: SUCCESS

```
{
  "code": "SUCCESS",
  "message": "OK",
  "data": {
    "userId": 10001,
    "username": "alice",
    "email": "alice@example.com",
    "role": "BEGINNER",
    "status": "ACTIVE",
    "createdAt": "2026-05-15T20:00:00+08:00"
  },
  "timestamp": "2026-05-15T20:12:00+08:00",
  "traceId": "req-auth-0007"
}
```

失败响应

HTTP 状态码: 401 Unauthorized

错误码: UNAUTHORIZED

```
{
  "code": "UNAUTHORIZED",
  "message": "未登录或令牌无效",
  "details": "Authorization 请求头缺失或格式错误",
  "timestamp": "2026-05-15T20:12:00+08:00",
  "traceId": "req-auth-0008"
}
```

7. 修改密码

接口: PATCH /api/v1/users/me/password

功能: 当前登录用户修改登录密码。

权限: 登录用户。

请求参数

请求头：

参数名	类型	必填	说明
Authorization	String	是	登录令牌，格式为 Bearer <JWT_TOKEN>
Content-Type	String	是	application/json

请求体：

参数名	类型	必填	说明
oldPassword	String	是	当前密码
newPassword	String	是	新密码，长度 8-64，需包含字母和数字

请求示例：

```
PATCH /api/v1/users/me/password
Authorization: Bearer <JWT_TOKEN>
Content-Type: application/json
```

```
{
  "oldPassword": "GitGuild2026",
  "newPassword": "GitGuild2026New"
}
```

成功响应

HTTP 状态码： 200 OK

业务码： SUCCESS

```
{
  "code": "SUCCESS",
  "message": "密码已修改",
  "data": {
    "changedAt": "2026-05-15T20:15:00+08:00"
  },
  "timestamp": "2026-05-15T20:15:00+08:00",
  "traceId": "req-auth-0009"
}
```

失败响应

HTTP 状态码: 400 Bad Request

错误码: PASSWORD_POLICY_VIOLATION

```
{
  "code": "PASSWORD_POLICY_VIOLATION",
  "message": "密码不符合安全策略",
  "details": "新密码长度至少为 8，且必须同时包含字母和数字",
  "timestamp": "2026-05-15T20:15:00+08:00",
  "traceId": "req-auth-0010"
}
```

8. 用户登出

接口: POST /api/v1/auth/logout

功能: 注销当前登录会话，使刷新令牌失效。

权限: 登录用户。

请求参数

请求头:

参数名	类型	必填	说明
Authorization	String	是	登录令牌，格式为 Bearer <JWT_TOKEN>

请求示例:

```
POST /api/v1/auth/logout
Authorization: Bearer <JWT_TOKEN>
```

成功响应

HTTP 状态码: 204 No Content

业务码: SUCCESS

响应体为空。

失败响应

HTTP 状态码: 401 Unauthorized

错误码: UNAUTHORIZED

```
{
  "code": "UNAUTHORIZED",
  "message": "未登录或令牌无效",
  "details": "访问令牌已失效",
  "timestamp": "2026-05-15T20:18:00+08:00",
  "traceId": "req-auth-0011"
}
```